

Parallel Bigram and Trigram Analysis: Performance Optimization with OpenMP

Lorenzo Cappetti
Department of Information Engineering
University of Florence

lorenzo.cappetti@edu.unifi.it

Abstract

In this work, I present a system for the generation and analysis of bigrams and trigrams over a corpus of approximately 1500 books. I evaluate the performance of the pipeline in both sequential and parallel versions, focusing on the impact of architectural design choices. The parallelization, based on OpenMP, combines thread local buffers, sharding, and a second version adopting a Structure of Arrays (SoA) layout to improve data locality and reduce allocations. The results show strong scalability up to 8–12 threads, followed by a plateau likely caused by memory bandwidth saturation during file loading, which represents the main bottleneck. The SoA version provides a small but consistent improvement. The analysis highlights that, in memory-bound workloads, scalability is limited more by file I/O than by computation.

1. Introduction

The objective of this project is to **quantitatively analyze** linguistic patterns in a large textual corpus through the extraction and study of **n-grams**. This type of analysis makes it possible to identify **frequencies**, **co-occurrences**, and **recurring structures** that are difficult to observe manually, with applications ranging from computational linguistics to the construction of predictive models and automatic text generation systems.

To support this analysis, the work follows a clear processing pipeline: text normalization and cleaning, tokenization, n-gram generation, and aggregation of counts. Alongside the linguistic aspect, particular attention is given to the computational side, evaluating both a *sequential approach* and *parallel strategies* aimed at improving scalability on multi-core systems.

The corpus consists of a large collection of public-domain texts; details about the source, selection criteria, and preprocessing are provided in the next section.

1.1. Dataset Description

The dataset employed consists of nearly 1500 digital books, totaling over 600 MB of text, an amount sufficient to guarantee robust statistics with low sensitivity to noise. All volumes were obtained from **Project Gutenberg**, one of the largest free digital libraries, which offers more than 70 000 works in the public domain or released under open licenses. The diversity of authors, genres, and historical periods makes this source particularly suitable for large-scale computational linguistic analyses.

The project repository includes the script `downloadBook.py`, which automates the download of texts from Project Gutenberg and their conversion to the `.txt` format. This ensures a standardized and easily reproducible acquisition process, overall, the resulting corpus provides a solid and consistent foundation for the study of n-grams.

1.2. Project Pipeline

The project follows a structured pipeline that transforms raw text files into aggregated n-gram statistics. The main stages are:

1. **Loading:** read the raw text files from disk and concatenate the content into memory buffers.
2. **Normalization and cleaning:** convert characters to a uniform representation (e.g., lowercase) and remove or replace irrelevant symbols, keeping only the information needed for tokenization.
3. **Tokenization:** split the cleaned text into tokens according to the chosen granularity (e.g., words or characters), producing a linear sequence of tokens.
4. **N-gram generation:** slide a window of size n over the token sequence to build n-grams (in this work, focusing on bigrams and trigrams), and count their occurrences.

5. **Aggregation:** merge partial counts into a global frequency table and compute the final statistics.
6. **Sorting and output:** sort n-grams by frequency and export the results in a readable format for inspection and benchmarking.

This same pipeline is implemented in both the sequential and parallel versions; the difference lies in how the workload is partitioned and how partial results are combined efficiently.

2. Text Normalization

Text normalization is a core step of the n-gram pipeline, whose purpose is to transform raw text into a **standardized** representation. By reducing superficial variability (e.g., casing, punctuation, or diacritics), normalization improves the **consistency** of tokenization and makes frequency statistics **comparable** across documents.

In this project, normalization consists of three main actions: **case folding** (e.g., converting to lowercase), **removing or replacing non-linguistic symbols** (punctuation, digits, and other special characters, depending on the tokenization rules), and **handling accented UTF-8 characters** by mapping them to a uniform form when appropriate.

Critical Aspects

Despite being conceptually simple, normalization introduces relevant challenges:

- **Performance impact:** the procedure is applied to a large amount of text, so even small inefficiencies can become a bottleneck.
- **Encoding robustness:** dealing with UTF-8 requires care to avoid corrupting multibyte characters and to ensure consistent mapping of accented symbols.
- **Trade-offs in cleaning rules:** aggressive filtering may remove useful information (e.g., apostrophes or hyphens), while conservative rules may leave noise that affects token boundaries and n-gram counts.

3. Tokenization

Tokenization converts the normalized text into a sequence of **tokens** that constitute the basic units for n-gram extraction. The definition of a token directly determines which patterns can be observed and therefore has a strong impact on the resulting frequency statistics.

In this project, tokenization is performed after normalization by scanning the text buffer and identifying token boundaries according to a set of rules. At a high level, the

procedure extracts **word-like units** by grouping contiguous alphabetic characters and splitting on separators such as whitespace and punctuation (as defined by the normalization step). The output is a linear stream of tokens that is then used to generate n-grams with a sliding window.

Critical Aspects

Tokenization presents several practical issues:

- **Boundary definition:** choices such as how to handle apostrophes, hyphens, or contractions affect token boundaries and can change n-gram counts significantly.
- **Robustness to noise:** residual punctuation or uncommon symbols may introduce empty tokens or unintended splits if not handled carefully.
- **Efficiency and memory:** token extraction may require scanning large buffers and storing token references; naïve approaches can cause excessive allocations and reduce cache locality.
- **Encoding considerations:** when UTF-8 characters are present, token boundary checks must avoid splitting multibyte sequences incorrectly.

4. N-gram Generation

Given the token sequence produced by tokenization, n-gram generation extracts contiguous sequences of length n by sliding a fixed-size window over the stream. In this work, we consider **bigrams and trigrams** ($n = 2$ and $n = 3$), i.e., sequences of two or three consecutive tokens. For each valid position i , the corresponding n-gram (e.g., (t_i, t_{i+1}) for bigrams) is generated and its frequency counter is updated accordingly.

Critical Aspects

- **Correctness at boundaries:** the procedure must handle document boundaries and avoid creating n-grams across unrelated segments (e.g., across books or across invalid/empty tokens).
- **Key representation:** storing n-grams as strings can be expensive; an efficient representation (e.g., string views, hashing, or integer IDs) strongly affects memory usage and speed.
- **High cardinality:** even with bigrams, the number of distinct keys can become large, requiring careful data-structure choices for fast updates.

5. Aggregation

Aggregation combines the counts produced during n-gram generation into a single global frequency table. Conceptually, it is a reduction: for each n-gram key, partial counters are summed to obtain the final total frequency used for analysis and reporting.

In our parallel design, aggregation is largely **implicit**, since threads update a shared **sharded** set of frequency tables; therefore, no explicit post-processing merge of thread-local maps is required. An explicit merge step would instead be needed in designs based on per-thread (or per-chunk) private maps.

Critical Aspects

- **Scalability:** whether aggregation is performed explicitly (merging private tables) or implicitly (concurrent updates), it can introduce contention; design choices determine whether it becomes a bottleneck.
- **Memory footprint:** explicit aggregation may require large intermediate structures (e.g., per-thread maps), while implicit aggregation may trade memory for synchronization and shared data-structure overhead.
- **Determinism and reproducibility:** the final counts must be independent of processing order; the aggregation strategy must guarantee correctness under parallel execution.

6. Sorting and Output

Once the global frequency table is computed, n-grams are sorted by frequency to produce a ranked list of the most common patterns. The results are then exported in a human-readable format to enable qualitative inspection and to support performance benchmarking across different implementations.

Critical Aspects

- **Cost of sorting:** sorting can become expensive when the number of distinct n-grams is large; it is typically performed after aggregation to avoid repeated work.
- **Output size:** writing very large rankings to disk can dominate runtime; practical reporting often focuses on top-*k* results or compressed outputs.
- **Tie handling:** when frequencies are equal, a consistent secondary ordering (e.g., lexicographic) improves reproducibility and comparability between runs.

7. Sequential Code

The sequential implementation provides the single-thread baseline for the whole pipeline. For each book, the

program performs: **file loading**, **Gutenberg cleaning**, **in-place normalization**, **tokenization**, and **n-gram counting**. Counts are first accumulated in hash maps (AoS) and then converted to a cache-friendly layout (SoA) for sorting and output.

7.1. Input Loading and Gutenberg Cleaning

All `.txt` files in the dataset folder are scanned and processed sequentially. Each book is loaded in **binary mode** by pre-sizing a `std::string` to the file size and reading the whole content in one operation. Before normalization, Project Gutenberg headers and footers are removed by searching for the **START/END markers** and shrinking the string **in-place** (single `memmove` + `resize`), avoiding additional allocations.

7.2. Sequential Tokenization

After normalization, the text is tokenized on a single thread using **pre-allocated buffers** to reduce reallocations:

- **Word tokenization:** tokens are extracted as `std::string_view` slices of the normalized buffer (zero-copy), splitting on whitespace. The resulting sequence is stored in a `std::vector<std::string_view>` with reserved capacity.
- **Character tokenization:** all non-whitespace characters are collected into a contiguous `std::vector<char>`, used to build character-level n-grams.

This design minimizes per-token allocations and provides contiguous memory access during the subsequent sliding-window passes.

7.3. N-gram Construction and Counting

N-grams are generated with a standard **sliding window** over the token buffers:

- **Word bigrams/trigrams:** for each position, consecutive tokens are concatenated into a reusable `std::string` key (cleared and reused at every step) and counted in global `std::unordered_map<std::string, size_t>` tables.
- **Character bigrams/trigrams:** similarly, character n-grams are generated by sliding over the character buffer. Keys use a fixed format (e.g., "`c1 c2`" and "`c1 c2 c3`") by resizing the same string and updating the relevant positions, reducing per-iteration overhead.

This phase is the most performance-sensitive part of the sequential pipeline, as it performs a large number of hash-map updates.

7.4. Finalization: SoA Conversion, Sorting, and Output

At the end of processing, each AoS hash map is converted into a **Structure of Arrays (SoA)** representation through a dedicated frequency counter: unique words are **interned** into a compact string pool and n-grams are stored as numeric IDs, improving cache locality and reducing duplicate string storage.

Finally, the n-grams are sorted by frequency and exported:

- Separate CSV files are produced for **word/character** and **bigrams/trigrams**.
- A statistics file reports the number of unique n-grams and timing information.

7.5. Sequential Benchmark

To obtain stable measurements, the program executes multiple runs:

- **Warm-up runs:** used to mitigate cold-cache effects and first-time allocations.
- **Measured runs:** used to compute mean, min/max, standard deviation, and coefficient of variation for both **wall-clock** time (including I/O) and **CPU** time.

Data structures are re-initialized between runs to ensure clean and comparable benchmarks. The resulting sequential timings provide the baseline for evaluating the scalability and overheads of the parallel implementations.

8. Parallel Code

The parallel implementation processes the corpus concurrently using **OpenMP**. The pipeline is identical to the sequential one (load → clean → normalize → tokenize → generate n-grams), but the workload is distributed across multiple threads and the global frequency tables are designed to support efficient concurrent updates.

8.1. Parallel Work Distribution with OpenMP

Books are processed in parallel through an OpenMP `parallel` region combined with a work-sharing loop:

- `#pragma omp parallel` creates a team of worker threads.
- `#pragma omp for schedule(dynamic)` assigns books to threads dynamically.

Dynamic scheduling is chosen because book sizes vary substantially; by assigning work in smaller chunks, it reduces load imbalance and limits idle time when some threads finish earlier than others.

8.2. Thread-local Processing: Loading, Cleaning, Normalization, Tokenization

Each thread independently performs the preprocessing steps on its assigned book:

- **File loading:** each book is read in binary mode by pre-sizing a `std::string` to the file size and reading the entire content in one operation.
- **Gutenberg cleaning:** headers and footers are removed in-place by searching for **START/END markers** and shrinking the buffer via `memmove` and `resize`.
- **In-place normalization:** the text is normalized directly in the same buffer (lowercasing, digit removal, punctuation handling, and basic UTF-8 accent mapping) to avoid intermediate copies.
- **Tokenization with thread-local buffers:** each thread maintains private buffers:
 - a `std::vector<std::string_view>` for **word tokens** (zero-copy slices of the normalized buffer);
 - a `std::vector<char>` for **character tokens** (all non-whitespace characters).

Buffers are **reserved** upfront (fixed capacities) to reduce reallocations and improve cache locality. Since these buffers are thread-local, normalization and tokenization require **no synchronization**.

8.3. Concurrent N-gram Counting with Sharded Maps

After tokenization, each thread generates word and character n-grams using a sliding window and updates global frequency tables. To minimize contention, the parallel version does not rely on a single shared hash map; instead, it uses a **sharded hash map** for each n-gram type (word bigrams, word trigrams, character bigrams, character trigrams).

8.3.1 Sharding and Locking Strategy

Each sharded table is split into a fixed number of shards (power of two), and a key is assigned to a shard via its hash:

$$\text{shard_idx} = h(\text{key}) \bmod \text{NUM_SHARDS}.$$

Each shard is protected by a **dedicated mutex**. As a consequence, threads only serialize when they update the *same* shard, while updates to different shards proceed concurrently.

8.3.2 Stable Key Storage with Per-shard Arenas

To avoid allocating a new heap string for each distinct n-gram and to keep references stable, each shard owns an **arena allocator**:

- new keys are copied into a contiguous arena block;
- the hash map stores `std::string_view` pointing into the arena memory;
- keys remain valid until the end of the program (no individual frees).

This design reduces allocator overhead and ensures that `string_view` keys do not dangle.

8.3.3 Insert-or-increment Operation

For each generated n-gram key, the update follows a standard **insert-or-increment** pattern:

- lock the target shard;
- if the key exists, increment its counter;
- otherwise, allocate the key in the shard arena and insert it with count 1.

To reduce per-iteration overhead, word n-gram keys are built using a reusable (thread-local) `std::string` buffer, while character n-grams use a small fixed-size stack buffer and are passed as `string_view`.

8.4. Aggregation, Sorting, and Output

In this design, **aggregation is implicit**: all threads update the same shared sharded tables, therefore no explicit merge of thread-local maps is required after processing.

After the parallel region completes, shards are traversed to materialize a global view of the results and to perform the final steps:

- **materialization**: all shard entries are collected into a single vector;
- **sorting**: n-grams are sorted by frequency;
- **export**: separate CSV files are produced for word/character and bigrams/trigrams, together with performance statistics.

8.5. AoS vs SoA Parallel Variants

Two parallel implementations are provided with identical algorithmic behavior but different data layouts:

- **AoS version (`parallel.cpp`)**: each shard is a cache-aligned struct containing `mutex` + `unordered_map` + `arena`. This layout groups shard components together and reduces indexing complexity.
- **SoA version (`parallel_soa.cpp`)**: shard components are stored in separate arrays (e.g., `mutexes[]`, `maps[]`, `arenas[]`); similarly, the arena allocator stores block metadata in separate vectors. The goal is to evaluate whether a SoA layout improves cache behavior for frequently accessed fields.

Both versions implement the same parallel pipeline and produce the same output format; differences are limited to internal memory organization and access patterns.

9. Results

In this section, we present the benchmark results obtained for the parallel versions of the system:

- `parallel`: parallel version with AoS data layout;
- `parallel_soa`: optimized parallel version with Structure of Arrays and arena allocator.

Both variants use OpenMP to distribute the dataset books across threads and exploit a global Sharded Map for concurrent n-gram insertion. Each configuration was executed 12 times to obtain reliable measurements, including 2 warm-up runs.

9.1. Experimental Setup

All benchmarks were executed on the same machine, whose specifications are reported below.

Category	Details
CPU	AMD Ryzen 5 7600X (6 cores / 12 threads, up to 5.46GHz)
RAM	32 GB DDR4
Cache	192 kB L1d + 192 kB L1i, 6 MB L2, 32 MB L3
Compiler	GCC 13.3.0 (g++), <code>-std=c++17</code>
Flags	<code>-O3 -march=native -DNDEBUG</code>
OpenMP	4.5 (<code>_OPENMP = 201511</code>)

Table 1. Hardware and software configuration used for benchmarking.

The project is built with **CMake** using `find_package(OpenMP REQUIRED)` and linking against `Threads::Threads` and `OpenMP::OpenMP_CXX`.

9.2. Choice of Structure of Arrays (SoA) and Achieved Benefits

In this project, a Structure of Arrays (SoA) representation was adopted to store n-grams and their corresponding counters. This choice stems from the fact that SoA improves data locality and reduces cache misses during update operations and partial result aggregation. Unlike the AoS approach, where each element contains all structured fields together, SoA separates components into independent vectors, enabling more linear memory scans and more uniform access patterns across threads. In the experimental evaluation, the use of SoA yielded slightly better performance compared to the non-SoA counterpart, especially at higher thread counts, where cache pressure is more significant. Although the performance gain is modest, it is consistent and measurable, reinforcing the choice of a modern and efficient data layout.

9.3. Scalability Analysis

The algorithm exhibits strong scalability up to approximately 8–12 threads, with a rapid decrease in execution time from over 120 s to about 16–20 s. In this region, parallelism is exploited extremely effectively thanks to the *embarrassingly parallel* nature of the problem: each book represents an independent unit of work and can be processed entirely within its assigned thread.

Table 2. Execution times for the `sequential` version.

Threads	Mean (s)	Min (s)	Max (s)	CV (%)
1	104.93	104.36	106.32	0.53

Table 3. Execution times for the `parallel` version.

Threads	Mean (s)	Min (s)	Max (s)	CV (%)
1	123.18	121.55	125.58	0.95
2	64.64	64.29	65.37	0.50
4	35.34	34.92	35.82	0.75
8	21.10	20.86	21.25	0.50
12	16.42	16.31	16.56	0.40
16	16.28	16.12	16.42	0.54
32	15.61	15.42	15.84	0.79
40	15.52	15.38	15.62	0.49
50	15.43	15.29	15.51	0.41
60	15.37	15.26	15.55	0.51
70	15.45	15.30	15.57	0.50
80	15.44	15.37	15.56	0.38
90	15.51	15.38	15.74	0.71
100	15.59	15.42	15.78	0.69

Beyond the 12–16 thread threshold, the curve flattens and the total execution time converges steadily to around 15–16 s. This plateau is not caused by computation, but

Table 4. Execution times for the `parallel_soa` version.

Threads	Mean (s)	Min (s)	Max (s)	CV (%)
1	124.27	122.83	125.71	0.75
2	65.24	64.54	66.02	0.76
4	35.19	34.75	35.58	0.64
8	20.93	20.78	21.15	0.55
12	16.29	16.21	16.43	0.42
16	16.02	15.95	16.12	0.30
32	15.58	15.44	15.76	0.61
40	15.40	15.31	15.47	0.34
50	15.37	15.25	15.53	0.46
60	15.38	15.28	15.60	0.65
70	15.38	15.25	15.50	0.58
80	15.36	15.24	15.54	0.58
90	15.47	15.38	15.55	0.37
100	15.56	15.44	15.84	0.70

rather by a bottleneck due to file loading into memory. In particular, reading files from the filesystem and copying their contents into each thread’s local buffer can quickly saturate the *memory bandwidth*. Since every thread must perform these preliminary operations before starting computation, RAM access becomes the dominant limiting factor of the entire process.

A relevant observation is that scalability continues to improve even beyond the number of physical cores (12): up to about **50 threads**, additional threads may hide part of the file I/O latency by exploiting CPU cycles that would otherwise remain idle. In this regime, virtual threads do not increase computational capacity, but help overlap memory operations with computation. However, beyond this point no further benefits are observed: memory bandwidth becomes fully saturated and the system reaches its theoretical limit, as evidenced by the nearly constant execution time between 50 and 100 threads.

The `parallel` (3) and `parallel_soa` (4) versions show very similar trends; the latter maintains a slight advantage due to improved data locality and reduced allocations, but both inevitably converge to the same memory-imposed limit.

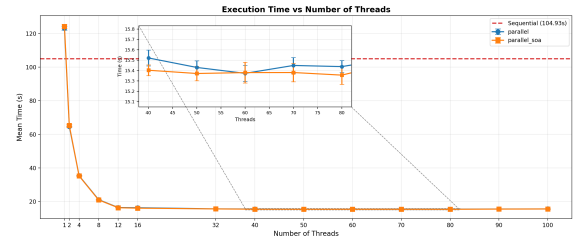


Figure 1. Total execution time as a function of the number of threads.

The plot highlights that:

- scalability is strong until file loading becomes dominant;
- *memory bandwidth* limits performance more than computation;
- virtual threads provide benefits up to about 50 threads, mainly by hiding I/O latency;
- beyond this point, execution becomes fully memory-bound, making further parallelism ineffective.

9.4. Speedup Analysis

The *Speedup vs Number of Threads* plot reports the achieved speedup with respect to sequential execution, compared against the ideal speedup $S_{\text{ideal}}(N) = N$. The latter represents the theoretical behavior of a perfectly scalable system, in which each additional thread proportionally reduces execution time.

The experimental curves of both `parallel` and `parallel_soa` versions instead show a rapid initial growth, reaching approximately $6.3\text{--}6.5\times$ speedup already at 12 threads, and then almost completely flattening. The speedup increase beyond 12–16 threads is minimal, and even at 50 or 100 threads it remains around $6.8\times$.

This behavior clearly indicates that the main bottleneck is not computation, but rather the **file loading into memory**. Each thread must perform three memory-intensive operations:

1. reading the file from the filesystem;
2. copying its contents into the thread-local buffer;
3. starting text processing.

This *file I/O and memory copy* phase rapidly saturates the **memory bandwidth**, preventing further improvements beyond 12–16 threads. As a result, the speedup curve flattens: the system becomes fully **memory-bound**.

A marginal improvement is still observed up to about **50 threads**. In this region, additional threads do not accelerate computation, but may help hide memory access latency. While some threads wait for file loading, others can progress, exploiting otherwise idle CPU cycles. Beyond 50 threads, memory bandwidth is completely saturated and no further improvements are possible.

The `parallel_soa` version achieves slightly higher speedup than `parallel`, thanks to improved data locality and reduced dynamic allocations enabled by the SoA layout. However, both implementations converge to the same memory-imposed limit.

In summary:

- speedup scales well up to 8–12 threads;

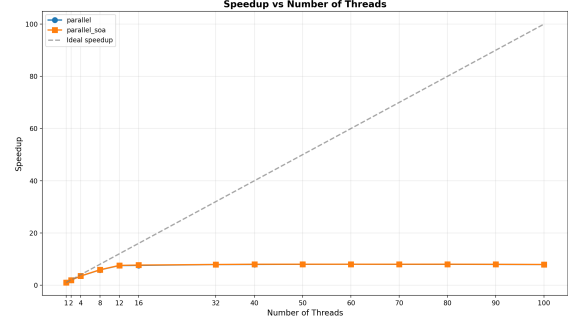


Figure 2. Speedup as a function of the number of threads, compared with ideal linear speedup.

- beyond this threshold, main memory becomes the dominant limiting factor;
- additional virtual threads marginally improve performance up to about 50 threads by hiding latency;
- beyond 50 threads, no benefit is observed: the system is fully memory-bound;
- the SoA version maintains a small but consistent advantage, though equally limited by the memory bottleneck.

9.5. Parallel Efficiency

Figure 3 shows the parallel efficiency as a function of the number of threads, computed as

$$E(p) = \frac{S(p)}{p} \times 100,$$

where $S(p)$ denotes the speedup achieved with p threads compared to sequential execution.

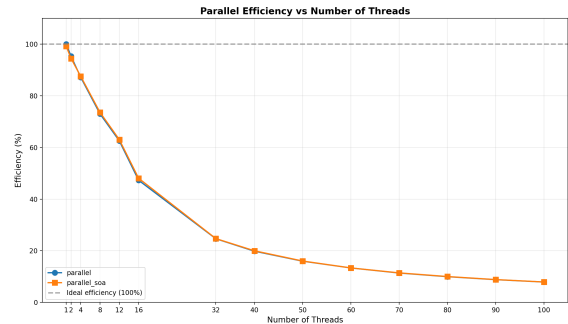


Figure 3. Parallel efficiency as a function of the number of threads.

At low parallelism levels (1–8 threads), efficiency remains very high and close to the ideal case. This indicates that the workload is perfectly distributable and synchronization overhead is negligible. Each thread processes an entire book independently, and the system behaves as an *embarrassingly parallel* problem.

Beyond 12–16 threads, efficiency rapidly declines: at 16 threads it drops below 50%, and at 32 threads it reaches approximately 25%. This behavior is not due to computation, but rather to the bottleneck introduced by file loading and memory bandwidth saturation. Since each thread must copy its assigned book into RAM before processing, main memory quickly becomes the limiting system resource.

From 40 up to 100 threads, efficiency stabilizes: in this region, adding more threads yields no improvement. Execution time is now exclusively limited by memory bandwidth, which cannot sustain such a high number of concurrent requests. Virtual threads only hide minor latencies, without increasing effective throughput.

The `parallel` and `parallel_soa` versions show almost identical trends, with a slight advantage for the SoA version due to improved data locality and reduced dynamic allocations. Nevertheless, both converge to the same physical limit imposed by the memory subsystem.

9.6. Comparison between `parallel` and `parallel_soa`

The `parallel_soa` version is slightly faster in almost all configurations, thanks to:

- **improved cache locality** provided by the SoA layout;
- **optimized arena allocator**, drastically reducing dynamic allocations;
- **more compact n-gram layout**, accelerating accesses during insertion.

Although the improvement is modest, it is systematic and more evident at low thread counts (1–4), where allocation overhead has a stronger impact.

9.7. Measurement Stability (Coefficient of Variation)

The **coefficient of variation (CV)** always remains below 1%, indicating:

- high performance stability,
- absence of significant run-to-run noise,
- consistent cache and data-structure behavior.

This stability strengthens the reliability of the measurements and the conclusions drawn regarding scalability.

10. Future Work

Several extensions could further improve both performance and analytical coverage. First, the current pipeline could be optimized by reducing I/O overhead (e.g.,

memory-mapped files or overlapping reading and processing) and by exploring lock-free or hybrid aggregation strategies to further limit contention. Second, alternative key representations (e.g., token ID encoding for word n-grams) could reduce memory footprint and speed up hash operations. Finally, the analysis could be extended beyond frequency counts by including additional linguistic statistics such as conditional probabilities, PMI-based association measures, or filtering by stop-words and document meta-data.

11. Conclusions

In this work, a system for the generation and processing of bigrams and trigrams over a corpus of approximately 1500 books (over 600 MB) was developed and analyzed. The main objective was to evaluate the system’s performance in both sequential and parallel versions, as well as the impact of design choices on efficiency and scalability.

The pipeline (reading, normalization, tokenization, and n-gram construction) naturally lends itself to parallelization, since each book constitutes an independent unit of work. The OpenMP-based implementation, combined with thread-local buffers and a global Sharded Map, enabled a stable, reproducible, and highly efficient pipeline.

The results demonstrate strong scalability up to 8–12 threads, with a significant reduction in execution time compared to the sequential version. Beyond this threshold, the primary limiting factor becomes **memory bandwidth**, as each thread must read and copy files into its local buffer before processing, rendering the system memory-bound. Total execution time stabilizes around 15–16 s.

A marginal improvement is observed up to approximately 50 threads, due to partial overlap of I/O and computation. Beyond this point, memory bandwidth is fully saturated, and additional threads do not provide further benefits.

The comparison between `parallel` and `parallel_soa` highlights a consistent but modest advantage of the SoA version, thanks to improved data locality and reduced dynamic allocations. However, both versions ultimately reach the same limit imposed by the memory subsystem.

In summary, this project demonstrates that it is possible to implement an efficient parallel pipeline for large-scale n-gram generation, provided that hardware constraints are considered. The employed techniques (sharding, thread-local buffers, SoA, arena allocator) reduce contention and maximize performance, highlighting that, in memory-bound workloads, scalability also depends on I/O and memory bandwidth. This work provides a useful model for future optimizations or extensions to larger datasets.

References

- [1] Project Gutenberg. *Free eBooks*. <https://www.gutenberg.org/>
- [2] OpenMP ARB. *OpenMP API Specification*, v4.5, 2018.
- [3] OpenAI. *ChatGPT (GPT-5-mini)*, 2025. <https://chat.openai.com/>